

Cool x Edgeless Systems Contrast

Binding verifiable AI-change evidence to a confidential Kubernetes workload, so a record proves not only what changed but which attested workload produced it.

DOCUMENT	Technical reference and integration specification
COMPONENT	cool-nwc/contrast
BASE VERSION	cool-nwc 3.0.0 (published); change set unreleased
CONTRAST READ AT	v1.25.0-pre, commit 3a93ecd
AUDIENCE	Engineers integrating, reviewing or operating the two systems
DATE	6 October 2026

STATUS OF THESE CLAIMS

Everything described here is implemented and tested: 152 automated tests pass, 33 of them specific to this integration, against certificates produced by Contrast's own code. No part of it has run on confidential hardware. The Kubernetes manifests have not been applied to a live cluster, and CoolL does not independently verify the TDX/SNP quote against Intel DCAP or AMD KDS. Section 10 states precisely what was and was not exercised.

1 • Summary

Contrast protects an AI workload while it runs. Cool produces independently verifiable evidence of what changed inside it. This integration joins the two so that a single receipt answers both questions at once.

The two systems address adjacent problems and neither addresses the other's. Contrast can prove that a pod executed as a confidential container on hardware matching a signed manifest, and can say nothing about which system prompt that pod was running. Cool can prove that a prompt changed, who changed it and that the record is unaltered, and can say nothing — by itself — about whether the process making that claim was protected while it ran.

The joint claim this integration makes checkable is therefore: **this AI change record was produced by a specific confidential workload whose execution environment can be independently attested.**

1.1 What was built

- `cool-nwc/contrast` — a new SDK entry point. `ContrastWorkload` reads the credential Contrast mounts into every pod; a browser-safe X.509, DER and manifest verifier checks it. **No new runtime dependency** was added to the SDK.
- An eighth verdict domain, `workload`, for attestation issued by an orchestrator rather than read from silicon. It reaches `pass` only against a Coordinator root certificate the verifier pinned itself.
- A cryptographic binding with two independent locks: the Cool signing key is derived from a secret Contrast releases only to an attested pod, and the pod's own mesh key signs a commitment to that Cool key.
- Deployment artefacts: a Kubernetes manifest, a Dockerfile, a deploy script following Edgeless's own command sequence, an example AI service, and an independent verifier an auditor can run with no cluster access.
- A conformance test suite whose fixtures are generated by **Contrast's own Go packages**, so the tests assert cross-implementation agreement rather than agreement with something Cool wrote for itself.

1.2 What makes the architecture work

The design rests on one observation about Contrast that is not obvious from its documentation and that changes the shape of any sensible integration: **Contrast's attestation is a credential, not a quote.** Section 3 develops this; section 4 builds the binding on it.

2 • The two systems

2.1 Cool in brief

Cool (Cryptographic Observability & On-chain Ledger) is an SDK and command line that turns an event into a self-contained receipt a third party can verify offline, years later, with no account and no trust in Cool. A receipt commits to what happened and proves it has not been altered; it deliberately does not carry the underlying data.

Layer	Mechanism	What it establishes
Canonicalization	RFC 8949 CDE canonical CBOR	The same logical record always produces the same bytes, across machines and implementations
Commitment	<code>binding_hash = mh:sha256(canonicalCBOR(core))</code>	A deterministic fingerprint of the whole record core
Signature	Hybrid ML-DSA-65 (FIPS 204) and Ed25519	Authenticity under both a post-quantum and a classical scheme; both must verify
Transparency	RFC 6962 Merkle log, signed tree head, inclusion proof	The record sits in an append-only log, so it cannot be removed from the middle of a history
Privacy	Salted commitments, <code>mh:sha256(salt bytes)</code>	Prompts, outputs and state are committed and discarded; receipts carry no plaintext

Cool records two kinds of thing through one pipeline, one key and one log: **execution evidence** (`cool.evidence.v1` — what the system did) and **change records** (`cool.change.v2` — what someone changed about it: a prompt edit, a model bump, a widened agent permission, a parameter or policy change). Sharing the log is the point: “what it was running” and “what it did” become provably one history rather than two systems to reconcile.

2.2 Contrast in brief

Contrast runs Kubernetes pods as confidential containers on Intel TDX or AMD SEV-SNP, built on Kata Containers and the Confidential Containers project. Its central component is the **Coordinator**, itself a confidential VM:

1. The operator writes a normal Kubernetes manifest and runs `contrast generate`, which computes each pod's identity, injects the Contrast plumbing, and writes `manifest.json` — the Coordinator's configuration, carrying a `Policies` map and the `ReferenceValues` each platform's attestation report must match.
2. `contrast set` hands that manifest to the Coordinator, which from then on admits a pod **only** if the pod's attestation report matches the reference values and its policy hash appears in `Policies`.
3. Each pod runs an **initializer** init container that performs an aTLS handshake to the Coordinator, carrying the pod's raw SNP or TDX report in an X.509 extension.
4. `contrast verify` lets a reader remotely attest the **Coordinator** against the manifest's reference values and write out its root certificate authority.

2.3 Why co-deployment is not an integration

Running Cool inside a Contrast pod without further work produces receipts that are cryptographically sound and say nothing about Contrast. The measurement is absent from the signed record; the signing key is whatever the operator configured; nothing stops a receipt produced on a laptop from being presented as the output of the confidential workload. The remainder of this document is about closing that gap.

3 • The key finding: Contrast attests by credential

Phala's dstack — Cool's existing confidential-compute backend — hands the workload a **raw vendor quote**. The workload passes it into the receipt and the verifier chains it to Intel DCAP or AMD KDS itself. Contrast's topology is different, and the difference is the integration.

When the Coordinator has verified a pod's attestation report, it issues that pod a **mesh certificate** and writes the claims it verified into the certificate as X.509 extensions, bound to a P-256 public key only that pod holds. From Contrast's own source, [coordinator/internal/meshapi/meshapi.go](#):

```
hostData := manifest.NewHexString(report.HostData())
entry, ok := state.Manifest().Policies[hostData]
if !ok {
    return nil, status.Errorf(codes.PermissionDenied,
        "policy hash %s not found in manifest", hostData)
}

extensions, err := report.ClaimsToCertExtension() // the measurements, into the cert
...
workloadSecretExtension, err := extension.ConvertExtension(
    extension.NewBytesExtension(oid.WorkloadSecretOID, []byte(entry.WorkloadSecretID)))
extensions = append(extensions, workloadSecretExtension)

cert, err := ca.NewAttestedMeshCert(dnsNames, extensions, peerPubKey)
```

The Coordinator's issuance path. peerPubKey is the pod's own key, from the aTLS handshake.

THE CONSEQUENCE

A Contrast credential is self-describing and offline-checkable. The measurements are inside the certificate, and the certificate chains to a root that can be pinned once. A verifier needs no Coordinator contact, no DCAP call and no network connection of any kind.

3.1 The extension map

Transcribed from [internal/oid/oid.go](#), [internal/attestation/tdx/extensions.go](#) and [internal/attestation/snp/extensions.go](#). Cool reads the following:

OID	Claim	Role in this integration
1.3.9901.2.2.14	TDX MRTD	The build-time measurement of the TD image
1.3.9901.2.2.18–21	TDX RTMR0–RTMR3	Runtime-extended registers; RTMR3 moves when the application does
1.3.9901.2.2.9	TDX MRSEAM	The TDX module that created the confidential VM
1.3.9901.2.2.15	TDX MRCONFIGID	The policy hash. <code>HostData()</code> is <code>MrConfigId[:32]</code>
1.3.9901.2.1.22	SNP MEASUREMENT	The SEV-SNP launch measurement
1.3.9901.2.1.23	SNP HOST_DATA	The policy hash on SEV-SNP
1.3.9901.3.1	WorkloadSecretID	The derivation label for the pod's sealed secret

The policy hash is the hinge of identity

The policy hash is the digest of the pod's **initdata document** and the key of the manifest's **Policies** map, documented in [internal/manifest/manifest.go](#) as “a map from policy hash (HOSTDATA) to policy entry”. So the certificate does not merely say “this pod was approved” — it states **which manifest entry authorised it**, which is a value an auditor can look up in a manifest they reviewed.

The insecure platform, and why it matters

Contrast supports development platforms with no confidential-computing hardware. On those, [internal/attestation/insecure/validator.go](#) returns **no certificate extensions at all**:

```
// ClaimsToCertExtension returns no extensions for insecure platforms.
func (r report) ClaimsToCertExtension() ([]pkix.Extension, error) {
    return nil, nil
}
```

Cool treats that absence as decisive. A credential with no measurements can never support a hardware claim, and the platform is decided by **which claim arc is present in the certificate**, never by configuration — so an operator cannot assert `intel-tdx` over a claimless certificate.

3.2 The runtime surface is a directory, not an API

By the time an application container starts, the initializer has written the following into the shared `contrast-secrets` volume mounted at `/contrast` (from [initializer/main.go](#)):

Path	Mode	Contents
<code>/contrast/tls-config/certChain.pem</code>	0444	The leaf mesh certificate followed by the intermediate CA
<code>/contrast/tls-config/key.pem</code>	0400	The pod's P-256 private key, PKCS#8 — generated inside the pod
<code>/contrast/tls-config/mesh-ca.pem</code>	0444	The mesh certificate authority

<code>/contrast/tls-config/coordinator-root-ca.0444</code>	0444	The Coordinator root certificate authority
<code>/contrast/secrets/workload-secret-seed</code>	0400	The workload secret, hex-encoded; present only when the manifest entry sets a WorkloadSecretID

Those files exist or the pod never got past its init container. Cool therefore needs no Contrast client library, no sidecar and no network call, which is why this integration adds no dependency and why the whole path can be exercised from fixtures on a laptop.

4 • The cryptographic binding

Cool binds on two sides and needs both. Either alone leaves a gap an attacker can walk through.

4.1 Sealing — the key cannot exist anywhere else

Cool derives its hybrid ML-DSA-65 + Ed25519 signing key from the Contrast workload secret, using HKDF-SHA256 with the derivation path as [info](#) so the record key and the log key are independent from one shared secret:

```
// src/contrast/workload.ts
async deriveKey(path: string): Promise<Uint8Array> {
  return hkdf(sha256, this.workloadSecret, utf8("cool/contrast/kms/v1"), utf8(path), 32);
}
```

The Coordinator releases that secret only to a pod whose attestation report matched the manifest, and derives it from the manifest's [WorkloadSecretID](#) — so it is **stable across restarts and manifest updates** and **different for every workload**. Three consequences follow:

- There is no signing key in the image, in a vault, or in CI. Outside the attested workload the key is not derivable at all, so there is nothing to steal, leak or rotate.
- Northwind Cipher cannot forge a record from a customer's deployment. Neither can the cluster operator, without passing attestation.
- The signing identity survives pod churn, so the audit trail stays continuous across restarts and rescheduling — which is what makes it usable in a real cluster.

A REAL BUG THIS SURFACED

Cool's key id was derived from the enclave measurement. Two Contrast pods built from the same image share an MRTD but receive different workload secrets, so they hold different keys — an image-derived id would have filed two distinct public keys under one name and silently broken any directory that merged them. Runtimes can now supply an unambiguous id; Contrast derives it from the policy hash. The test suite found this, not review.

4.2 Certifying — the key belongs to this workload

Sealing proves the key came from an attested pod. It does not prove **which** pod. For that, the pod's mesh key signs a commitment to the Cool public key:

```
statement = { key_id, ed25519_pub, ml_dsa_pub, issued_at }
message   = "cool/contrast/key-binding/v1" || canonicalCBOR(statement)

key_binding      = mh:sha256(message)
binding_signature = ECDSA(message, /contrast/tls-config/key.pem)
```

`key_id` and `issued_at` are **inside** the signed statement rather than beside it, so a binding cannot be lifted onto a different Cool key or silently re-dated. This is the structural analogue of a quote's `report_data`, with the Coordinator standing where the silicon stands.

4.3 The complete chain

```
AI change (prompt · model · policy · permission · parameter)
  | committed as salted hashes; plaintext never leaves the workload
  ▼
cool.receipt.v2
  | record.runtime.workload ← the Contrast identity, INSIDE the signed core
  ▼
hybrid signature  ML-DSA-65 + Ed25519 over canonicalCBOR(core) || binding_digest
  ▼
RFC 6962 transparency log  inclusion proof + signed tree head
  ▼
attestation.workload.binding_signature
  | ECDSA by the pod's mesh key over a commitment to the signing key
  ▼
mesh certificate  MRTD · RTMRs · policy hash · WorkloadSecretID
  ▼
Coordinator intermediate → Coordinator root CA
  | pinned by the reader, from their own: contrast verify
  ▼
Coordinator CVM  remotely attested against the manifest's reference values
  ▼
confidential execution  Intel TDX / AMD SEV-SNP
```

4.4 Why substitution fails

Every row below is an automated test in `tests/contrast.test.ts`. The design intent is that no single swap succeeds and no combination of swaps succeeds either.

Attack	What breaks	Domain that reports it
Attach workload A's receipt to workload B's credential	The binding signature verifies only under A's certificate	<code>workload</code>
Also rewrite the sealed identity so the two agree	The identity is inside the signed core, so the record signature breaks	<code>signature</code> , <code>binding</code>

Edit the AI change itself	<code>binding_hash</code> and the record signature break; the credential stays genuine, so the verdict names which guarantee failed	<code>binding</code> , <code>signature</code>
Flip one byte in the certificate	Chain verification fails	<code>workload</code>
Forge or re-date the binding signature	ECDSA verification fails; <code>issued_at</code> is inside the signed statement	<code>workload</code>
Present a Coordinator you stood up yourself	The chain does not reach the pinned root, and the record's sealed <code>coordinator_root</code> digest names the mismatch explicitly	<code>workload</code>
Remove the credential, or remove the sealed identity	A half-binding is a hard failure, not a weaker binding	<code>workload</code> , <code>signature</code>
Rebuild the image	The MRTD changes and a register pin fails; previously issued receipts keep verifying	<code>workload</code>
Replay an inclusion proof from another record	The audit path no longer reconstructs the signed tree head	<code>inclusion</code>

5 • Receipt format

Two fields were added. Both are optional in the **encoding** sense, not merely nullable: when a deployment has no orchestrator-issued credential the keys are absent from the canonical CBOR entirely.

BACKWARD COMPATIBILITY WAS A HARD CONSTRAINT

cool-nwc@3.0.0 is already published. Receipts issued by it must continue to hash, sign and verify to exactly the same bytes, which rules out a workload: null field — adding one would change the binding hash of every receipt ever produced. A test asserts that the key is absent rather than null.

```
{
  "record": {
    "runtime": {
      "tee_vendor": "intel-tdx",
      "mode": "hardware",
      "tee_quote": null,
      "workload": {
        "schema": "cool.workload.v1",
        "platform": "contrast",
        "tee": "intel-tdx",
        "policy_hash": "hex:f04dfba7...",
        "workload_secret_id": "default/ai-service",
        "workload_name": "ai-service",
        "sans": ["ai-service", "10.42.0.11"],
        "registers": { "mrtid": "hex:...", "rtmr0": "hex:...", "rtmr3": "hex:...", "mrseam": "hex:..." },
        "registers_digest": "mh:sha256:...",
        "coordinator_root": "mh:sha256:..."
      }
    }
  }
}
```

```

    "manifest_digest": "mh:sha256:..."
  }
},
"attestation": {
  "workload": { // ← in the envelope
    "schema": "cool.workload.attestation.v1",
    "cert_chain": "-----BEGIN CERTIFICATE-----...",
    "key_binding": "mh:sha256:...",
    "bound_key_id": "cool-contrast-enclave-f04dfba7241e",
    "binding_signature": "base64:...",
    "binding_alg": "1.2.840.10045.4.3.2",
    "issued_at": "2026-10-05T14:52:32.000Z"
  }
}
}

```

The identity goes in the signed core; the credential in the envelope. The core already commits to everything that matters about the credential, and the credential is large.

5.1 Why the split

Every field of `WorkloadIdentityV1` is recomputable from the credential in the envelope. That is what makes the pairing checkable: swap the credential and the recomputed identity stops matching the signed one; edit the signed identity and the record signature breaks. The source of both halves is the same object — `ContrastWorkload` returns them together, because only it can produce them consistently, having read the identity out of the very certificate whose key makes the signature.

That also keeps the evidence plane platform-agnostic: Cool's core never parses a certificate, so Contrast support is an integration layer rather than a dependency of the SDK's centre.

6 • The verifier

`verifyEvidence` takes bytes and returns a structured verdict — never a bare boolean. It never throws, because a verifier is routinely looking at bytes an adversary chose. Verification requires no network access for any domain.

6.1 Eight domains

Domain	Question	Reaches pass when
<code>binding</code>	Does the commitment match the record?	Recomputation agrees. Pure arithmetic
<code>signature</code>	Was it signed by the named key?	Both ML-DSA-65 and Ed25519 verify
<code>inclusion</code>	Is it in the log?	The audit path reconstructs a validly signed tree head
<code>witnesses</code>	Did anyone independent co-sign the head?	At least the threshold of <code>external: true</code> co-signatures verify

attestation	Does the vendor quote chain to Intel/AMD/NVIDIA?	A configured verifier accepted a non-simulated root
enclave	Is the quote bound to this record and this key?	Quote digest, measurement and report_data all agree
workload	Was this produced by the Contrast workload it claims?	The credential chains to a Coordinator root the verifier pinned, and the identity matches the signed core
anchor	Did the head exist before a given Bitcoin block?	The recomputed commitment equals a real block's merkle root

workload is a separate domain rather than an additional way to pass **enclave** because the trust argument is genuinely different. A verified quote is checked arithmetic against a silicon vendor. A passing workload credential means an authority the reader attested separately vouched for the hardware. Both are hardware-rooted; one is transitive. Keeping them apart lets a verdict state which of the two it actually checked.

6.2 The workload domain's six checks

Ordered so that failures are diagnostic rather than merely negative. Each closes a specific substitution.

1. **Shape** — the credential parses and carries a leaf certificate.
2. **Key binding** — the binding statement is recomputed from the receipt's **own** key directory and the signature over it checked under the leaf certificate's public key. This is what stops a genuine credential being stapled to a record signed by some other key; the attacker would need the pod's mesh key.
3. **Identity** — the identity recomputed from the certificate must equal the identity sealed inside the signed record core.
4. **Chain** — the chain must verify to a Coordinator root the **verifier** pinned. Not a root the receipt carried: a pod can mint its own certificate authority, so an unpinned chain proves only self-consistency.
5. **Manifest** — the policy hash must be one the pinned manifest authorises, and the sealed manifest digest must match the pinned bytes.
6. **Registers** — the attestation registers must match what was pinned.

6.3 The honesty rules

These are the reason to trust the rest of the verdict.

Situation	Reported as	Rationale
No Coordinator root pinned by the verifier	absent	"The binding is valid and self-consistent, but with no pinned Coordinator root it is REPORTED, not verified." A pod can supply any CA
Contrast insecure platform	simulated	A real signature by a real Coordinator over a workload whose hardware nobody checked

Pinned root, real claims, identity agrees	pass	The full chain held
Contrast receipt, quote-shaped domains	attestation: mock, enclave: absent	There is no quote. The <code>enclave</code> detail names the credential route rather than claiming the record was not produced in a TEE

`requireHardware` is satisfied by **either** a verified vendor quote **or** a workload credential that chained to a pinned Coordinator root, and the verdict always names which route was taken, so the distinction survives into the audit trail instead of being flattened at the point of decision.

WHAT IS DELIBERATELY NOT DONE

`getQuote()` returns null under Contrast. Synthesising a quote, or re-labelling the certificate as one, would be the single genuinely dishonest move available in this integration. It is not made, and a test asserts the quote-shaped domains report their own absence.

6.4 Runtime status

Cool derives a runtime status from evidence rather than configuration — `REAL`, `UNVERIFIED`, `SIMULATED`, `UNAVAILABLE` or a failure. That machinery is quote-shaped, so it needed a credential branch; without one, a healthy Contrast pod on SEV-SNP would have been reported as `EVIDENCE INCOMPLETE` because SNP has no RTMRs to be non-zero, or sent the operator to configure a quote verifier for a quote that does not exist.

Observation	State	Why that is the ceiling
Contrast <code>insecure</code> platform	<code>SIMULATED</code>	No measurements in the credential
Credential checked against the root the pod itself was given	<code>UNVERIFIED</code>	A pod cannot establish its own Coordinator — the root arrived with the credential, so this proves coherence, not provenance
Credential checked against a root the operator pinned out of band	<code>REAL</code>	The authority was attested independently
Credential did not check out	<code>ATTESTATION FAILED</code>	The channel closes rather than downgrading

7 • Integration surface

7.1 Adopting it in an application

Adoption is a runtime swap, not a rewrite. No call site changes.

```
import { Cool } from "cool-nwc";
import { ContrastWorkload } from "cool-nwc/contrast";

const runtime = await ContrastWorkload.open({ requireConfidential: true });
const cool = new Cool({ applicationId: "refund-agent", runtime });

await cool.change({
  kind: "prompt",
  ref: "billing/refund-agent#system",
  before: "Refund when the policy allows.",
  after: "Refund when the policy allows. Escalate above $500.",
  actor: { id: "user:priya@bank.example", method: "session" },
});
```

`requireConfidential` makes an insecure platform fatal at start-up rather than something discovered at audit time. A production deployment should set it and never unset it.

7.2 Verifying, as an auditor

```
cool verify receipts.json \
  --coordinator-root verify/coordinator-root-ca.pem \
  --manifest manifest.json \
  --require-hardware
```

Or programmatically, with the full set of pins a regulated deployment would use:

```
import { verifyEvidence } from "cool-nwc";

const verdict = await verifyEvidence(receipt, {
  coordinatorRootCA: readFileSync("verify/coordinator-root-ca.pem", "utf8"),
  expectedManifest: readFileSync("manifest.json"),
  expectedRegisters: { mrtid: "a27518a5..." }, // the image you approved
  allowedPolicyHashes: ["f04dfba7..."], // the pods you approved
  requireHardware: true,
});
```

7.3 Public surface added

Export	Kind	Purpose
<code>ContrastWorkload</code>	class (Node)	Reads <code>/contrast</code> , derives the sealed key, produces the credential

<code>verifyWorkloadDomain</code>	function	The workload domain, usable standalone
<code>parseCertificates</code> , <code>verifyChain</code> , <code>verifyEcdsa</code>	functions	Browser-safe X.509 for callers doing their own checks
<code>parseManifest</code> , <code>workloadPolicyHashes</code> , <code>policyFor</code>	functions	Reading a Contrast manifest as a pinning input
<code>identityFromCertificate</code> , <code>bindingMessage</code>	functions	Identity extraction and the signed binding bytes
<code>oid</code>	namespace	The OID table, transcribed from Contrast's source
<code>AttestationSource</code>	interface	What the evidence plane requires of a runtime; <code>DstackClient</code> also satisfies it
<code>CoolL.change()</code>	method	The governance half of the model, previously only on <code>CoolTee</code>

7.4 Dependencies

None added. The X.509, DER, ASN.1 and PEM handling is written against the existing `@noble/curves` and `@noble/hashes` dependencies. A reader can audit roughly 900 lines more cheaply than they can justify another supply-chain claim. The parser is deliberately strict — indefinite lengths, non-minimal lengths, multi-byte tags and trailing bytes are all rejected, because a lenient parser in front of a signature check is how certificate confusion bugs happen.

The verifier half stays browser-safe on purpose: an auditor should be able to verify a Contrast-bound receipt in a browser tab. Only `ContrastWorkload`, which reads files, is Node-only.

8 • Deployment

8.1 The three lines that adopt it

The pod specification is a normal Kubernetes `Deployment` plus three things. `contrast generate` reads it and injects the Contrast plumbing — the `contrast-initializer` init container, the `contrast-service-mesh` sidecar, the shared `/contrast` volume and the platform-specific runtime class — so none of that is hand-written here.

```
runtimeClassName: contrast-cc          # marks the pod for contrast generate

metadata:
  annotations:
    contrast.edgeless.systems/workload-secret-id: default/ai-service

env:
  - name: COOL_CONTRAST_ROOT
    value: /contrast
```

Line	Why it is required
<code>runtimeClassName:</code> <code>contrast-cc</code>	<code>contrast generate</code> only touches pods carrying this marker, and replaces the bare name with the platform-specific handler for the cluster
<code>workload-secret-id</code> annotation	Tells the Coordinator to release a workload secret. Without it Cool has nothing to seal its key to , and refuses to start with that message and the fix. The value also lands in the certificate as extension <code>1.3.9901.3.1</code> , so it is part of the verified identity
<code>COOL_CONTRAST_ROOT=/contrast</code>	Tells Cool to use the Contrast credential. One environment variable is the whole adoption step

8.2 The deployment sequence

`examples/contrast/deploy.sh` is deliberately just Edgeless's own command sequence in order, plus one Cool-specific step. It should be read before it is run.

```
kubectl apply -f coordinator.yml          # 1 • the Coordinator CVM
contrast generate --reference-values <platform> k8s/ai-service.yaml
kubectl create configmap contrast-manifest \      # 3 • Cool-specific
  --from-file=manifest.json=manifest.json
contrast set -c <coordinator>:1313 k8s/ai-service.yaml
kubectl apply -f k8s/ai-service.yaml
contrast verify -c <coordinator>:1313          # 6 → verify/coordinator-root-ca.pem
```

Step 3 is the only addition. Cool seals the manifest's **digest** into every receipt so each record states which manifest governed it, and the pod needs the bytes to compute that digest. Step 6 is what makes every receipt from the deployment checkable: it remotely attests the Coordinator against the manifest's reference values and writes the root an auditor pins.

8.3 Infrastructure required

- A Kubernetes cluster with bare-metal Intel TDX or AMD SEV-SNP, or AKS with Confidential Containers.
- The Contrast node-installer and runtime class applied to the cluster.
- The `contrast` CLI, matching the release the node-installer came from.
- A reproducible workload image. Contrast measures it into the TD, the measurement lands in the certificate, and Cool seals it into every receipt — so a non-reproducible build makes “unmodified code” unverifiable. Pin the base image by digest and commit a lockfile.

THERE IS NO KEY TO PROVISION

The workload image carries no signing key and needs none: Cool derives its key from the Contrast workload secret at start-up. That removes the entire key-distribution and key-rotation problem from the deployment, and is one of the main practical reasons the integration is worth building.

9 • Trust model

9.1 Trusted

Component	What is assumed, and why
Intel TDX / AMD SEV-SNP	The hardware root. Cool adds nothing here and claims nothing about it
The Contrast Coordinator	It decides which pods are admitted and issues every credential. A compromised Coordinator can certify an arbitrary workload. This is Contrast's own trust assumption; Cool inherits it and narrows it by requiring the reader to attest the Coordinator themselves
The reader's own <code>contrast verify</code> run	Pinning the root is the step that makes everything downstream meaningful
ML-DSA-65, Ed25519, SHA-256/384, ECDSA P-256/P-384	Standard primitives, via <code>@noble/*</code>

9.2 Not trusted

The cluster operator, the Kubernetes control plane, the node operating system, the container registry, the network, Northwind Cipher, Edgeless Systems' servers, and the verifier's own network connection — verification is entirely offline, and a test asserts that no network call is made.

9.3 Threats considered

Threat	Status
Receipt substitution across workloads	Closed. See section 4.4
Key substitution — a credential attesting a different key	Closed. <code>key_binding</code> is recomputed from the receipt's own key directory
Identity substitution — relabelling the workload	Closed. The identity is inside the signed core
Confused deputy — Cool signing on behalf of another workload	Closed. The key derives from this pod's workload secret, and the key id derives from the policy hash so two pods on one image cannot collide
Metadata spoofing — asserting TDX in configuration	Closed. The platform is decided by which claim arc is present in the certificate
Stale measurements	Open, documented. The certificate's claims are from the issuing handshake, so a long-lived pod's credential reflects its state at admission rather than now. Contrast's one-year certificate lifetime bounds this; a deployment wanting tighter bounds should rotate pods
Binding replay	Open, documented. Reusing an old binding requires already holding both the Cool signing key and the mesh key — that is, having compromised the pod. Record freshness comes from the transparency log and the optional Bitcoin anchor, not from the binding
Workload migration	Handled and tested. A rescheduled pod gets a new certificate and the same workload secret, so the signing identity is continuous and the certificate serial changes. Receipts from before and after both verify
Container escape	Out of scope. It would yield the workload secret and the mesh key, hence the ability to sign new records, and is not detectable from a receipt. Contrast's isolation is the control
Kubernetes privilege escalation	The Cool container needs no privileges, and Contrast mounts <code>/contrast</code> read-only into every container except the initializer
Secret leakage	The workload secret is read once at start-up and never written, logged or transmitted. Cool never signs application data with the mesh key — one statement, at start-up
Time-of-check/time-of-use	The credential is obtained once, before any caller data exists, and its identity is in every signed core. A record cannot be produced before the binding exists
Verifier trust	None required: no network, and <code>pass</code> needs a reader-supplied root
Parser differentials	The DER reader rejects indefinite and non-minimal lengths, multi-byte tags and trailing bytes; and the parse is cross-checked against Contrast's own encoder (section 10.1)
Denial of service against the verifier	Closed. Two polynomial-backtracking regular expressions in the PEM path were replaced with linear scans; see section 10.4

9.4 Known limitations

1. **Cool does not re-verify the TDX/SNP quote.** It trusts the Coordinator's verdict as expressed in the certificate. The extensions actually contain enough to reconstruct the quote — including the PCK certificate

chain at [1.3.9901.2.2.46](#) — so independent verification is possible. It is not implemented, and is the highest-value next step (section 11).

2. **Stale measurements and binding replay**, as above.
3. **No confidential-GPU path yet.** Cool's [GpuAttestationRef](#) exists and Contrast supports confidential GPUs; the two are not joined.
4. **The transparency log is in memory by default.** For a log that spans restarts — which is what makes ordering and completeness provable rather than a hundred trees of size one — mount a volume and pass [FileLog](#) from [cool1-nwc/node](#), ideally on a Contrast encrypted volume via the [secure-pv](#) annotation.
5. **This is not production-ready.** It is tested; it is not deployed.

10 · Testing and conformance

10.1 Cross-implementation conformance

Cool does **not** write its own fixtures for Contrast's certificate format. [tools/contrast-fixture/main.go](#) runs inside a Contrast checkout and calls Contrast's own packages:

Contrast package used	What it supplies
internal/seedengine — New , RootCAKey , GenerateMeshCAKey , DeriveWorkloadSecret	The Coordinator's real key hierarchy and workload-secret derivation
internal/ca — New , NewAttestedMeshCert	The real cross-signed CA and the real mesh certificate template
internal/attestation/tdx — Report.ClaimsToCertExtension	The real TDX claim to X.509 extension encoding
internal/oid — WorkloadSecretOID	The real workload-secret extension

So a test asserting “Cool recovered this MRTD” asserts agreement between **Contrast's Go encoder and Cool's TypeScript decoder**. That caught details documentation alone would not have surfaced: the extension payload is doubly wrapped because Go's [asn1.Marshal\(\[\]byte\)](#) produces an OCTET STRING inside the [extnValue](#) OCTET STRING; Contrast's CA keys are P-384 while workload keys are P-256; and the chain is cross-signed, with the mesh CA self-signed under the intermediate key.

WHAT THE FIXTURES DO NOT PROVE

The TDX quote whose claims the fixture certificates carry is constructed in software. A real Coordinator obtains that quote from the pod's aTLS handshake and verifies it against Intel DCAP before issuing anything. These fixtures prove that Cool parses, binds to and enforces Contrast's credential format correctly. They prove nothing about hardware.

10.2 Test results

```
$ npm test
  tests 152
  pass  152
  fail  0
```

Group	Tests	Covers
Conformance	4	TDX claims against Contrast's encoder; policy hash present in the manifest; chain to the genuine root but not a rogue one; insecure platform carries no claims
Happy path	4	Sealing inside Contrast; identity inside the signature; <code>requireHardware</code> pinned versus unpinned; insecure never reported as confidential
Substitution	6	Cross-workload credential; full double swap; rogue Coordinator; half-binding; flipped certificate byte; forged and re-dated binding
Pinning	3	Policy allow-list; manifest digest; register drift on a rebuilt image
Lifecycle	3	Restart preserves the sealed key; missing <code>WorkloadSecretID</code> ; not-a-Contrast-pod
Governance	3	Four change kinds in one log; tampered change; replayed inclusion proof
Runtime status	3	<code>UNVERIFIED</code> from inside the pod, <code>REAL</code> only with a pinned root, <code>SIMULATED</code> on insecure; the verdict's runtime label
Hardening	4	Malformed identity, malformed attestation claim, a manifest the verifier itself supplied badly, adversarial PEM
Compatibility	1	A non-Contrast receipt is byte-identical and the domain is <code>absent</code>
Contrast total	33	Of 152 across the whole SDK

10.3 Verified by hand, end to end

- The example service starts against a materialised fixture pod, reports its Contrast identity, and seals receipts through `/change` and `/infer`.
- `examples/contrast/verify.mjs` returns `2/2 verified` with the correct Coordinator root and exit code `1` with a rogue one.
- `cool verify --coordinator-root --manifest --require-hardware` passes on a Contrast receipt and fails with an actionable reason when the root is omitted; exit codes gate correctly for CI.
- `npm run verify:package` packs the tarball, installs it into a clean project, and confirms the `cool-nwc/contrast` subpath resolves under both `nodenext` and `bundler` module resolution.

10.4 Found by the pipeline, not by review

Two findings are worth recording because they were missed by inspection.

- `CodeQL js/polynomial-redos` flagged two regular expressions in the PEM path — the block parser in

`src/contrast/x509.ts` and the chain shape-check in `src/phala/structure.ts`. The verifier parses PEM an attacker chose, so polynomial backtracking there is a denial-of-service vector against anyone verifying untrusted receipts. Both are now linear `indexOf` scans, with a regression test that puts roughly 400 KB of adversarial PEM through the verifier and requires it to finish; it does, in about 80 ms.

- The repository's secret-scanning gate rejected the original fixture layout, which committed `key.pem` files. Rather than weaken the rule, the fixtures were restructured as base64 DER inside one JSON file, and `tests/support/contrast-pod.ts` writes a pod layout into a temporary directory at run time — so the tests still exercise the real file paths and the real PEM parsing.

10.5 Not tested, because it needs hardware

- A real Intel TDX or AMD SEV-SNP machine.
- A real Coordinator CVM, a real aTLS handshake, real DCAP or KDS collateral.
- `contrast generate`, `contrast set` and `contrast verify` against a live cluster.
- The Kubernetes manifests in `examples/contrast/k8s/` have **not** been applied to a cluster. They are written against Contrast's current generator and are unvalidated.
- The `Dockerfile` carries a placeholder base-image digest.

11 • The highest-value next step

Reconstruct and independently verify the TDX/SNP quote from the mesh certificate's own extensions.

The extensions already carry the whole quote: the `TdQuoteBody`, the ECDSA signature at `1.3.9901.2.2.24`, the attestation key at `.25`, and the **PCK certificate chain** at `.46`. If Cool reassembles that into a wire-format quote and verifies it against Intel DCAP collateral, two things change:

1. The `attestation` domain can reach `pass` on a Contrast receipt, rather than honestly reporting `mock`.
2. The Coordinator leaves the trusted set **for the hardware claim**. It still decides admission, but a reader no longer has to take its word that the silicon was real.

That would make a Cool × Contrast receipt verifiable **to silicon, offline, without trusting the Coordinator** — materially stronger than either project offers alone, and the natural technical centre of a joint reference architecture.

12 · Source references

Every Contrast behaviour described in this document is cited to github.com/edgelessys/contrast, read at commit [3a93ecd](https://github.com/edgelessys/contrast/commit/3a93ecd), version [1.25.0-pre](https://github.com/edgelessys/contrast/releases/tag/1.25.0-pre).

Claim	File
Extension OIDs	internal/oid/oid.go
TDX claim numbering	internal/attestation/tdx/extensions.go
SNP claim numbering	internal/attestation/snp/extensions.go
HostData is <code>MrConfigId[:32]</code> ; SNP HostData	internal/attestation/tdx/validator.go , internal/attestation/snp/validator.go
Insecure platform issues no extensions	internal/attestation/insecure/validator.go
Mesh certificate issuance; workload-secret release	coordinator/internal/meshapi/meshapi.go
Cross-signing CA; P-384 CA keys	internal/ca/ca.go , internal/seedengine/seedengine.go
Runtime file paths; the P-256 pod key	initializer/main.go
Policies keyed by policy hash (HOSTDATA)	internal/manifest/manifest.go
Policy hash is the initdata digest	internal/initdata/initdata.go , cli/cmd/verify.go
verify/ output filenames	cli/cmd/common.go , cli/cmd/verify.go
Injected initializer, sidecar and volume	internal/kuberesource/parts.go , mutators.go
Workload-secret annotation	internal/kuberesource/constants.go

Edgeless documentation consulted: *The manifest*, *Secrets & recovery*, and *Service mesh* at docs.edgeless.systems/contrast.

12.1 Licensing

Contrast and Cool are both under the Business Source License 1.1, with separate licensors and separate terms.

No Contrast source is vendored into the Cool repository. [tools/contrast-fixture/main.go](#) is Cool-authored code that imports Contrast's packages by reference and compiles only inside a Contrast checkout; nothing it depends on is redistributed. The runtime integration in [src/contrast/](#) depends on no Contrast code at all — it reads files and parses certificates — so it carries no Contrast licence obligation.

12.2 Reproducing this

```
git clone https://github.com/Northwind-Cipher/cool-sdk.git
cd cool-sdk && npm install

npm run typecheck
npm test           # 152 tests, 33 of them Contrast
npm run demo:contrast # the whole integration end to end, no cluster, ~90s
npm run verify:package # pack → clean install → run → typecheck
```

To regenerate the conformance fixtures from Contrast's own code, see [tools/contrast-fixture/README.md](#).